EJERCICIO 4 - Búsqueda en profundidad

Grafo: ver ej4_grafo.png (datos de la Tabla 3).
Bosque abarcador: ver ej4_bosque.png.

En los órdenes, V es la cantidad de vértices y E la cantidad de aristas.

---

1. LISTA DE ADYACENCIAS

```
    Asuncion       -> Buenos_Aires(1600), Curitiba(800), Porto_Alegre(700)
    Buenos_Aires   -> Asuncion(1600), Montevideo(200), Porto_Alegre(1250)
    Curitiba       -> Rio_de_Janeiro(1500)
    Montevideo     -> Buenos_Aires(200), Porto_Alegre(1000)
    Porto_Alegre   -> Montevideo(1000)
    Rio_de_Janeiro -> Curitiba(1500)
```

Los adyacentes se recorren en este orden (el de las columnas de la tabla).

---

2. BÚSQUEDA EN PROFUNDIDAD DESDE ASUNCIÓN

Num(v) es el orden en que se visita cada vértice.

```
    Asu -> BA    BA no visitado         ARBOL      Num(BA) = 2
    BA  -> Asu   Asu en la recursión    RETROCESO
    BA  -> Mvd   Mvd no visitado        ARBOL      Num(Mvd) = 3
    Mvd -> BA    BA en la recursión     RETROCESO
    Mvd -> PoA   PoA no visitado        ARBOL      Num(PoA) = 4
    PoA -> Mvd   Mvd en la recursión    RETROCESO  (terminan PoA y Mvd)
    BA  -> PoA   PoA ya terminó         AVANCE     (termina BA)
    Asu -> Cur   Cur no visitado        ARBOL      Num(Cur) = 5
    Cur -> Rio   Rio no visitado        ARBOL      Num(Rio) = 6
    Rio -> Cur   Cur en la recursión    RETROCESO  (terminan Rio y Cur)
    Asu -> PoA   PoA ya terminó         AVANCE     (termina Asu)
```

Num(Asu) = 1. Desde Asunción se llega a los 6 vértices, así que el bosque
tiene un solo árbol.

Orden de visita:
    Asuncion, Buenos_Aires, Montevideo, Porto_Alegre, Curitiba, Rio_de_Janeiro

Tipos de arcos:
- Árbol (5): Asu-BA, BA-Mvd, Mvd-PoA, Asu-Cur, Cur-Rio.
- Retroceso (4): BA-Asu, Mvd-BA, PoA-Mvd, Rio-Cur. Van a un antecesor.
- Avance (2): BA-PoA y Asu-PoA. Van a un descendiente ya visitado.
  Se comprueba con Num(v) < Num(w) <= Num(v) + descendientes(v):
  BA: 2 < 4 <= 2 + 2. Asu: 1 < 4 <= 1 + 5.
- Cruzados: ninguno, porque no hay arcos entre la rama de Buenos Aires y la
  de Curitiba.

Como hay arcos de retroceso, el grafo tiene ciclos.

---

3. recorridoEnProfundidad

Lenguaje natural:
Se marca el vértice como visitado y se le pasa al consumidor. Para cada arco saliente cuyo destino no fue visitado, se hace lo mismo desde ese destino. El conjunto de visitados se pasa como parámetro a los llamados recursivos.
- Desde un origen: solo visita lo alcanzable desde él.
- Todo el grafo: empieza por el primer vértice y, si quedan vértices sin visitar, arranca otro árbol desde el siguiente. Usa un solo conjunto de visitados para todo el recorrido.

El método default de la interfaz llamaba al recorrido desde cada vértice con un conjunto de visitados nuevo cada vez, y repetía vértices. Por eso se sobrescribe en DirectedGraphAlgorithms.

Precondiciones: la común (el grafo está creado). Si el origen no existe, no se visita nada.

Postcondiciones:
- El grafo no cambia.
- El consumidor recibe cada vértice visitado una sola vez, en orden de profundidad.

Pseudocódigo:

```
recorridoEnProfundidad(grafo, consumidor)
Comienzo
    visitados <- conjunto vacío
    PARA CADA v EN vertices(grafo) HACER
        SI v NO ESTA EN visitados ENTONCES
            bpf(v, visitados, consumidor)
        FIN SI
    FIN PARA
Fin

bpf(actual, visitados, consumidor)
Comienzo
    visitados.agregar(actual)
    consumidor.procesar(actual)
    PARA CADA sucesor EN sucesores(actual) HACER
        SI sucesor NO ESTA EN visitados ENTONCES
            bpf(sucesor, visitados, consumidor)
        FIN SI
    FIN PARA
Fin
```

Orden del tiempo de ejecución:
Cada vértice se visita una vez y cada arista se mira una vez, lo que daría $O(V + E)$. Pero en esta implementación, para obtener las adyacencias de un vértice primero hay que buscarlo con un Comparable, y eso cuesta $O(V)$. Como se hace una vez por vértice, el total es $O(V^2 + E)$. Con un acceso directo al vértice (buscarlo por clave en el mapa) sería $O(V + E)$.